

ASSOCIATION ANALYSIS

Association analysis

- Methodology useful for discovering interesting relationships hidden in large data sets
- Uncovered relationships can be represented in the form of association rules

Association Rule Mining

- Given a set of transactions, find rules that will predict the occurrence of an item based on the occurrences of other items in the transaction

Market-Basket transactions

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Example of Association Rules:

{Diaper} → {Beer}

(strong relationship exists between the sale of diapers and beer)

- Discovering patterns from a large transaction data set can be computationally expensive
- Some of the discovered patterns are potentially spurious because they may happen simply by chance
- Association analysis is also applicable to other application domains such as bioinformatics, medical diagnosis, Web mining and scientific data analysis

Basic Terminology

Binary representation:

TID	Bread	Milk	Diapers	Beer	Eggs	Cola
1	1	1	0	0	0	0
2	1	0	1	1	1	0
3	0	1	1	1	0	1
4	1	1	1	1	0	0
5	1	1	1	0	0	1

- Let $I = \{i_1, i_2, i_3, \dots, i_d\}$ be the set of all items in a market basket data
- $T = \{t_1, t_2, t_3, \dots, t_N\}$ be the set of all transactions
- Each transaction t_i contains a subset of items chosen from I

- Itemset

- Collection of zero or more items
- Example: {Milk, Bread, Diaper}

- k -itemset

- An itemset that contains k items

- **Transaction width**

- Number of items present in a transaction

- **Support count ()**

- Number of transactions that contain a particular itemset

$$\sigma(X) = |\{t_i | X \subseteq t_i, t_i \in T\}|$$

- E.g. $\sigma(\{\text{Milk, Bread, Diaper}\}) = 2$

Association Rule

- An implication expression of the form $X \rightarrow Y$, where X and Y are disjoint itemsets
- Strength of an association rule can be measured in terms of its Support and Confidence

Support

- Determines how often a rule is applicable to a given data set
- Fraction of transactions that contain both X and Y

Confidence

- Determines how frequently items in Y appear in transactions that contain X

$$\begin{aligned}\text{Support, } s(X \rightarrow Y) &= \frac{\sigma(X \cup Y)}{N} \\ \text{Confidence, } c(X \rightarrow Y) &= \frac{\sigma(X \cup Y)}{\sigma(X)}\end{aligned}$$

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Consider the rule: $\{\text{Milk, Diaper}\} \Rightarrow \{\text{Beer}\}$

$$s = \frac{\sigma(\text{Milk, Diaper, Beer})}{N} = \frac{2}{5} = 0.4$$

$$c = \frac{\sigma(\text{Milk, Diaper, Beer})}{\sigma(\text{Milk, Diaper})} = \frac{2}{3} = 0.67$$

Why Use Support and Confidence?

- A rule that has very low support may occur simply by chance
- A low support rule is also likely to be uninteresting from a business perspective because it may not be profitable to promote items that customers seldom buy together
- For these reasons, support is often used to eliminate uninteresting rules
- Support also has a desirable property that can be exploited for the efficient discovery of association rules

- **Confidence** measures the reliability of the inference made by a rule
- For a given rule $X \rightarrow Y$, the higher the confidence, the more likely it is for Y to be present in transactions that contain X
- Confidence also provides an estimate of the conditional probability of Y given X

Formulation of Association Rule Mining Problem

- Definition (Association Rule Discovery):

Given a set of transactions T , find all the rules having support $\geq \text{minsup}$ and confidence $\geq \text{minconf}$

(where minsup and minconf are the corresponding support and confidence thresholds)

- A brute-force approach for mining association rules is to compute the support and confidence for every possible rule
- This approach is prohibitively expensive because there are exponentially many rules that can be extracted from a data set
- Total number of possible rules extracted from a data set that contains d items is

$$R = 3^d - 2^{d+1} + 1$$

- Even for the small data set, this approach requires us to compute the support and confidence for large number of rules
- More than 80% of the rules are discarded after applying $\text{minsup} = 20\%$ and $\text{minconf} = 50\%$, thus making most of the computations become wasted
- To avoid performing needless computations, it would be useful to prune the rules early without having to compute their support and confidence value

Solution:

- Decouple the support and confidence requirements
- Support of a rule $X \rightarrow Y$ depends only on the support of its corresponding itemset, $X \cup Y$

```
{Beer, Diapers} → {Milk}, {Beer, Milk} → {Diapers},  
{Diapers, Milk} → {Beer}, {Beer} → {Diapers, Milk},  
{Milk} → {Beer, Diapers}, {Diapers} → {Beer, Milk}.
```

- These rules have identical support because they involve items from the same itemset {Beer, Diapers, Milk}

Mining Association Rules

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Example of Rules:

$\{\text{Milk, Diaper}\} \rightarrow \{\text{Beer}\}$ ($s=0.4$, $c=0.67$)

$\{\text{Milk, Beer}\} \rightarrow \{\text{Diaper}\}$ ($s=0.4$, $c=1.0$)

$\{\text{Diaper, Beer}\} \rightarrow \{\text{Milk}\}$ ($s=0.4$, $c=0.67$)

$\{\text{Beer}\} \rightarrow \{\text{Milk, Diaper}\}$ ($s=0.4$, $c=0.67$)

$\{\text{Diaper}\} \rightarrow \{\text{Milk, Beer}\}$ ($s=0.4$, $c=0.5$)

$\{\text{Milk}\} \rightarrow \{\text{Diaper, Beer}\}$ ($s=0.4$, $c=0.5$)

Observations:

- All the above rules are binary partitions of the same itemset: {Milk, Diaper, Beer}
- Rules originating from the same itemset have identical support but can have different confidence
- Thus, we may decouple the support and confidence requirements

- Common strategy adopted by many association rule mining algorithms is to decompose the problem into two major subtasks:
- **Frequent Itemset Generation**
 - whose objective is to find all the itemsets that satisfy the minsup threshold. These itemsets are called frequent itemsets
- **Rule Generation**
 - whose objective is to extract all the high-confidence rules from the frequent itemsets found in the previous step. These rules are called strong rules

The computational requirements for frequent itemset generation are generally more expensive than those of rule generation

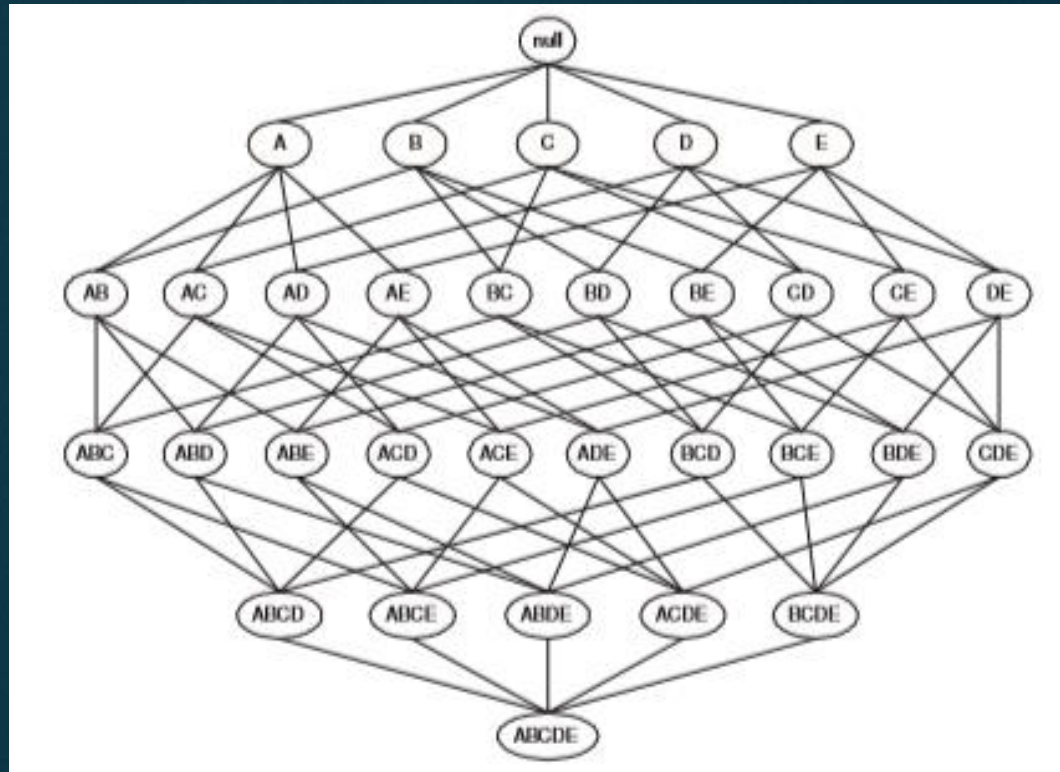
Frequent Itemset Generation

A lattice structure can be used to enumerate the list of all possible itemsets

Let $I = \{A, B, C, D, E\}$

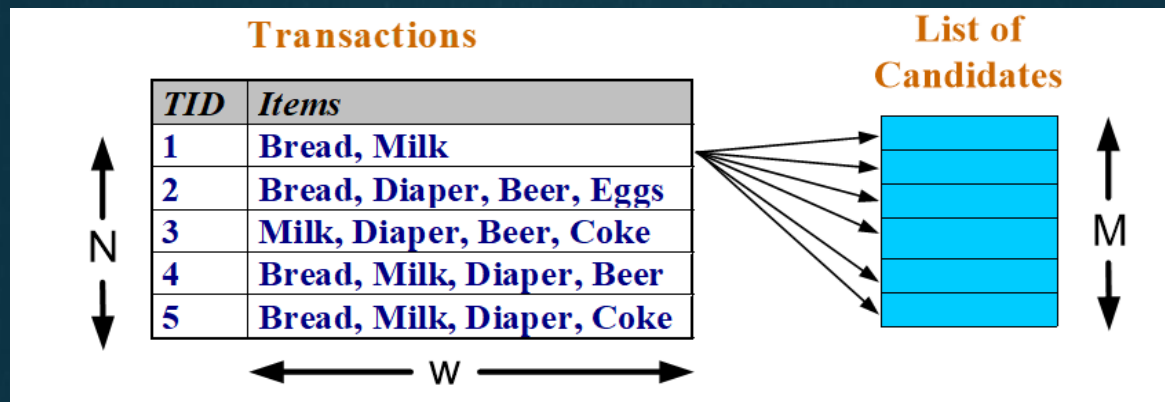
Given d items, there
are 2^d

possible candidate
itemsets



Brute-force approach:

- Each itemset in the lattice is a **candidate** frequent itemset
- Count the support count of each candidate by scanning the database



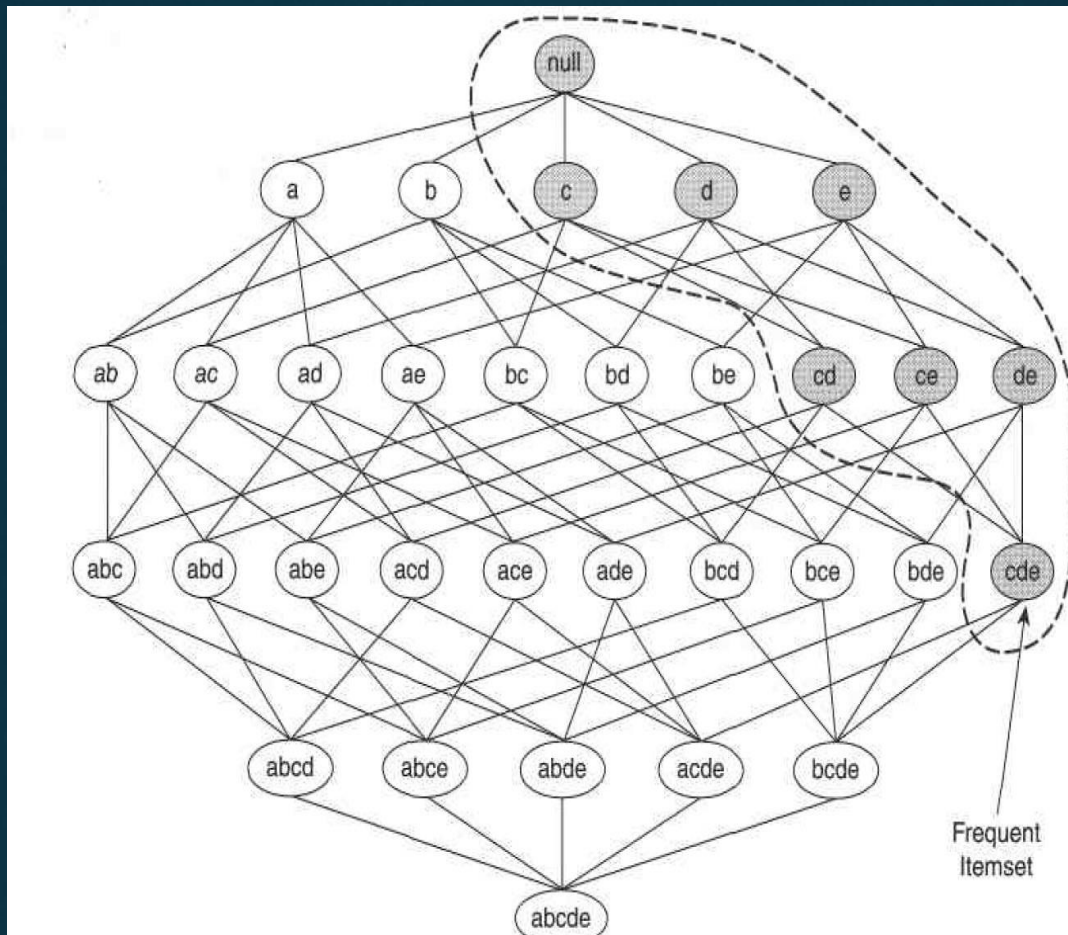
- Match each transaction against every candidate
- Complexity $\sim O(NMw) \Rightarrow$ **Expensive** since $M = 2^d$!!!

Ways to reduce computational complexity

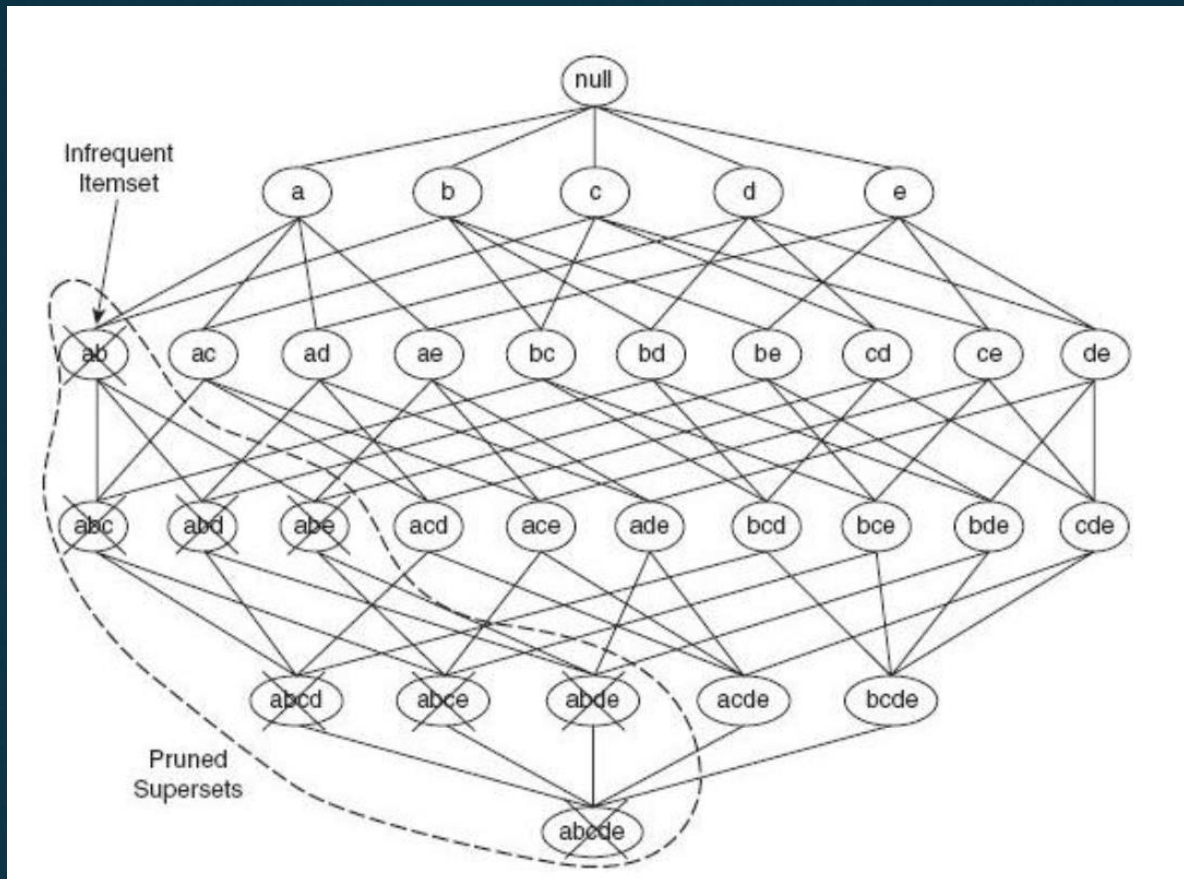
- Reduce the number of candidate itemsets(M)
- The Apriori Principle is an effective way to eliminate some of the candidate itemsets
- Reduce the number of comparisons
- Use more advanced data structures either to store the candidate itemsets or to compress the data set

The Apriori Principle

- Used to Reduce the number of candidate itemsets
- If an itemset is frequent, then all of its subsets must also be frequent
- Suppose $\{c, d, e\}$ is a frequent itemset
- Any transaction that contains $\{c, d, e\}$ must also contain its subsets, $\{c, d\}, \{c, e\}, \{d, e\}, \{c\}, \{d\}$, and $\{e\}$
- As a result, if $\{c, d, e\}$ is frequent, then all subsets of $\{c, d, e\}$ must also be frequent



- Conversely, if an itemset such as $\{a, b\}$ is infrequent, then all of its supersets must be infrequent too
- the entire subgraph containing the supersets of $\{a, b\}$ can be pruned immediately once $\{a, b\}$ is found to be infrequent
- This strategy of trimming the exponential search space based on the support measure is known as **Support-based pruning**



- Such a pruning strategy is made possible by a key property of the support measure
- Anti-monotone property of the support measure:
The support for an itemset never exceeds the support for its subsets

Monotonicity and Antimonotonicity Property

- Let I be a set of items, and $J = 2^I$ be the power set of I
- A measure f is **monotone** (or upward closed) if

$$\forall X, Y \in J : (X \subseteq Y) \longrightarrow f(X) \leq f(Y)$$

i.e. if X is a subset of Y , then $f(X)$ must not exceed $f(Y)$

- f is **anti-monotone** (or downward closed) if

$$\forall X, Y \in J : (X \subseteq Y) \longrightarrow f(Y) \leq f(X),$$

i.e. if X is a subset of Y , then $f(Y)$ must not exceed $f(X)$

Illustrating Apriori Principle

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Beer, Bread, Diaper, Eggs
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Bread, Coke, Diaper, Milk



Items (1-itemsets)

Item	Count
Bread	4
Coke	2
Milk	4
Beer	3
Diaper	4
Eggs	1

Minimum Support = 3

If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3$$

$$6 + 15 + 20 = 41$$

With support-based pruning,

$$6 + 6 + 4 = 16$$

Illustrating Apriori Principle

TID	Items
1	Bread, Milk
2	Beer, Bread, Diaper, Eggs
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Bread, Coke, Diaper, Milk

Item	Count
Bread	4
Coke	2
Milk	4
Beer	3
Diaper	4
Eggs	1

Items (1-itemsets)



Itemset
{Bread, Milk}
{Bread, Beer }
{Bread, Diaper}
{Beer, Milk}
{Diaper, Milk}
{Beer, Diaper}

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3

If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3 \\ 6 + 15 + 20 = 41$$

With support-based pruning,

$$6 + 6 + 4 = 16$$

Illustrating Apriori Principle

TID	Items
1	Bread, Milk
2	Beer, Bread, Diaper, Eggs
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Bread, Coke, Diaper, Milk

Item	Count
Bread	4
Coke	2
Milk	4
Beer	3
Diaper	4
Eggs	1

Items (1-itemsets)



Itemset	Count
{Bread, Milk}	3
{Beer, Bread}	2
{Bread, Diaper}	3
{Beer, Milk}	2
{Diaper, Milk}	3
{Beer, Diaper}	3

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3

If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3$$

$$6 + 15 + 20 = 41$$

With support-based pruning,

$$6 + 6 + 4 = 16$$

Illustrating Apriori Principle

TID	Items
1	Bread, Milk
2	Beer, Bread, Diaper, Eggs
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Bread, Coke, Diaper, Milk

Item	Count
Bread	4
Coke	2
Milk	4
Beer	3
Diaper	4
Eggs	1

Items (1-itemsets)



Itemset	Count
{Bread, Milk}	3
{Bread, Beer}	2
{Bread, Diaper}	3
{Milk, Beer}	2
{Milk, Diaper}	3
{Beer, Diaper}	3

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3

If every subset is considered,
 ${}^6C_1 + {}^6C_2 + {}^6C_3$
 $6 + 15 + 20 = 41$
 With support-based pruning,
 $6 + 6 + 4 = 16$



Itemset
{ Beer, Diaper, Milk}
{ Beer, Bread, Diaper}
{Bread, Diaper, Milk}
{ Beer, Bread, Milk}

Triplets (3-itemsets)

Illustrating Apriori Principle

TID	Items
1	Bread, Milk
2	Beer, Bread, Diaper, Eggs
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Bread, Coke, Diaper, Milk

Item	Count
Bread	4
Coke	2
Milk	4
Beer	3
Diaper	4
Eggs	1

Items (1-itemsets)



Itemset	Count
{Bread, Milk}	3
{Bread, Beer}	2
{Bread, Diaper}	3
{Milk, Beer}	2
{Milk, Diaper}	3
{Beer, Diaper}	3

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3

If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3 \\ 6 + 15 + 20 = 41$$

With support-based pruning,

$$6 + 6 + 4 = 16$$



Triplets (3-itemsets)

Itemset	Count
{ Beer, Diaper, Milk}	2
{ Beer, Bread, Diaper}	2
{Bread, Diaper, Milk}	2
{Beer, Bread, Milk}	1

Illustrating Apriori Principle

TID	Items
1	Bread, Milk
2	Beer, Bread, Diaper, Eggs
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Bread, Coke, Diaper, Milk

Item	Count
Bread	4
Coke	2
Milk	4
Beer	3
Diaper	4
Eggs	1

Items (1-itemsets)



Itemset	Count
{Bread, Milk}	3
{Bread, Beer}	2
{Bread, Diaper}	3
{Milk, Beer}	2
{Milk, Diaper}	3
{Beer, Diaper}	3

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3

If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3 \\ 6 + 15 + 20 = 41$$

With support-based pruning,

$$6 + 6 + 4 = 16 \\ 6 + 6 + 1 = 13$$



Triplets (3-itemsets)

Itemset	Count
{ Beer, Diaper, Milk}	2
{ Beer, Bread, Diaper}	2
{Bread, Diaper, Milk}	2
{Beer, Bread, Milk}	1

Algorithm 6.1 Frequent itemset generation of the *Apriori* algorithm.

```
1:  $k = 1$ .
2:  $F_k = \{ i \mid i \in I \wedge \sigma(\{i\}) \geq N \times \text{minsup} \}$ .    {Find all frequent 1-itemsets}
3: repeat
4:    $k = k + 1$ .
5:    $C_k = \text{apriori-gen}(F_{k-1})$ .    {Generate candidate itemsets}
6:   for each transaction  $t \in T$  do
7:      $C_t = \text{subset}(C_k, t)$ .    {Identify all candidates that belong to  $t$ }
8:     for each candidate itemset  $c \in C_t$  do
9:        $\sigma(c) = \sigma(c) + 1$ .    {Increment support count}
10:    end for
11:  end for
12:   $F_k = \{ c \mid c \in C_k \wedge \sigma(c) \geq N \times \text{minsup} \}$ .    {Extract the frequent  $k$ -itemsets}
13: until  $F_k = \emptyset$ 
14:  $\text{Result} = \bigcup F_k$ .
```

- F_k : frequent k -itemsets
- C_k : candidate k -itemsets

Algorithm

- Let $k=1$
- Generate $F_1 = \{\text{frequent 1-itemsets}\}$
- Repeat until F_k is empty
 - ◆ **Candidate Generation:** Generate C_{k+1} from F_k
 - ◆ **Candidate Pruning:** Prune candidate itemsets in C_{k+1} containing subsets of length k that are infrequent
 - ◆ **Support Counting:** Count the support of each candidate in C_{k+1} by scanning the DB
 - ◆ **Candidate Elimination:** Eliminate candidates in C_{k+1} that are infrequent, leaving only those that are frequent $\Rightarrow F_{k+1}$

Candidate Generation

List of requirements for an effective candidate generation procedure:

- It should avoid generating too many unnecessary candidates(atleast one of its subsets is infrequent)
- It must ensure that the candidate set is complete(no frequent itemsets are left out)
- It should not generate same candidate itemset more than once

Candidate Generation: Brute-force method

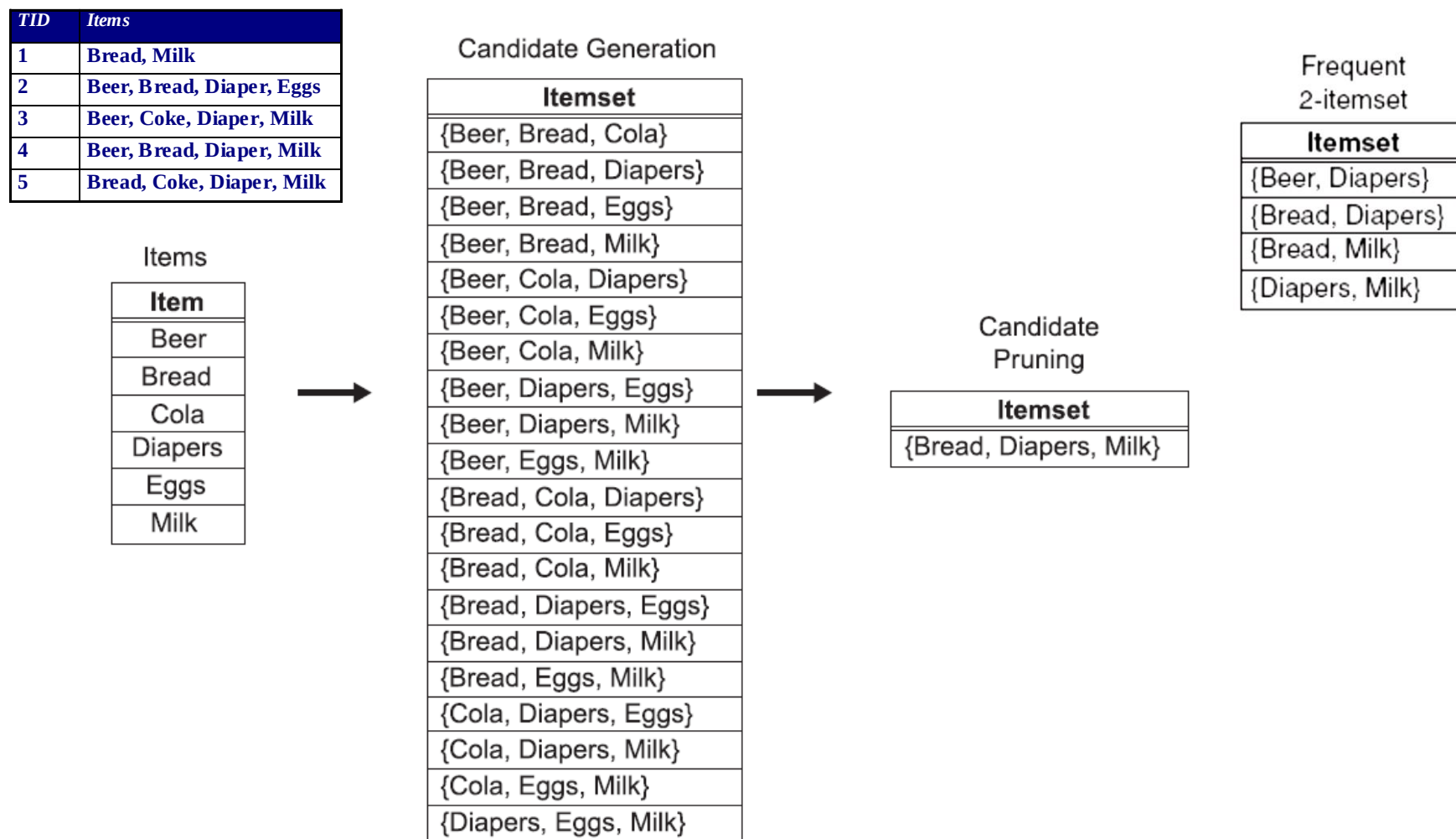


Figure 5.6. A brute-force method for generating candidate 3-itemsets.

Candidate Generation: Merge Fk-1 and F1 itemsets

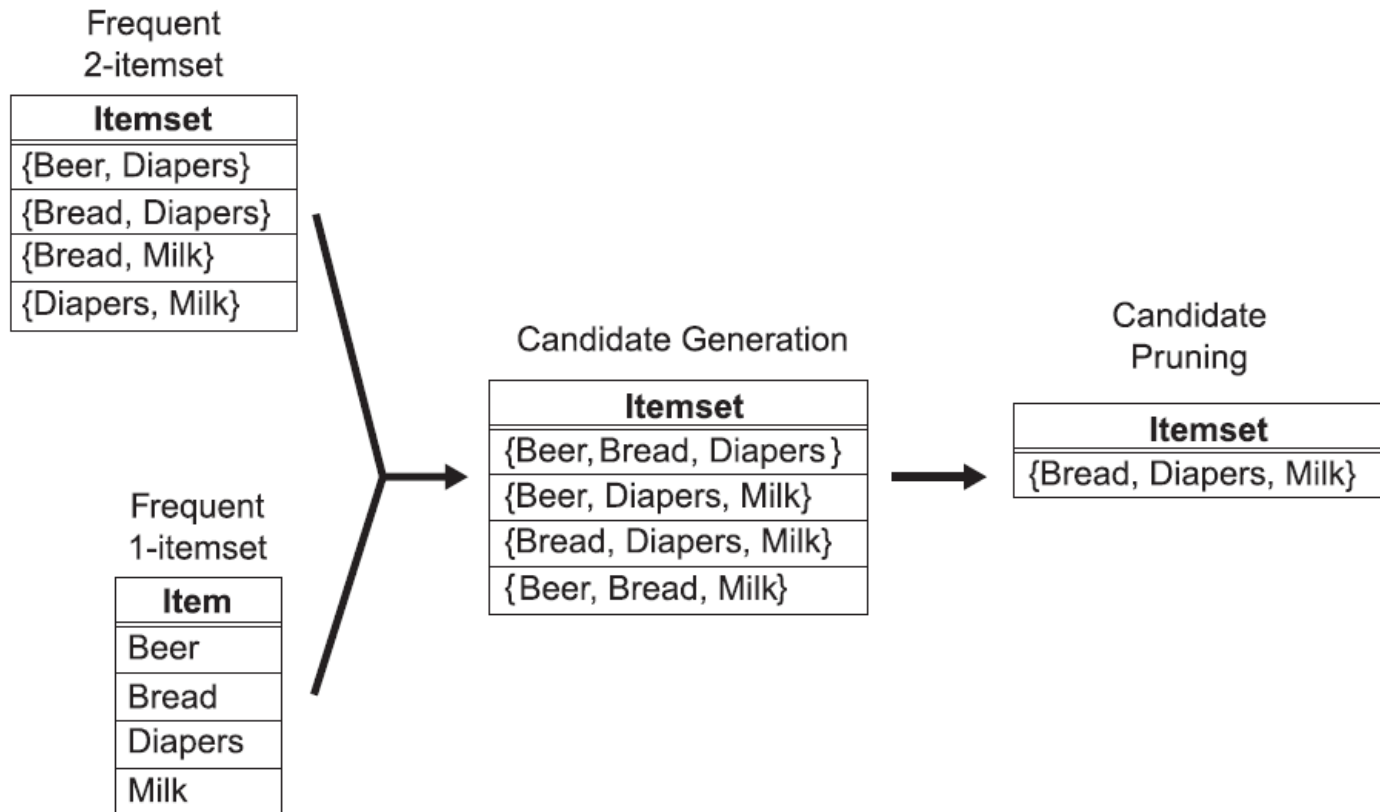


Figure 5.7. Generating and pruning candidate k -itemsets by merging a frequent $(k-1)$ -itemset with a frequent item. Note that some of the candidates are unnecessary because their subsets are infrequent.

Candidate Generation: Fk-1 x Fk-1 Method

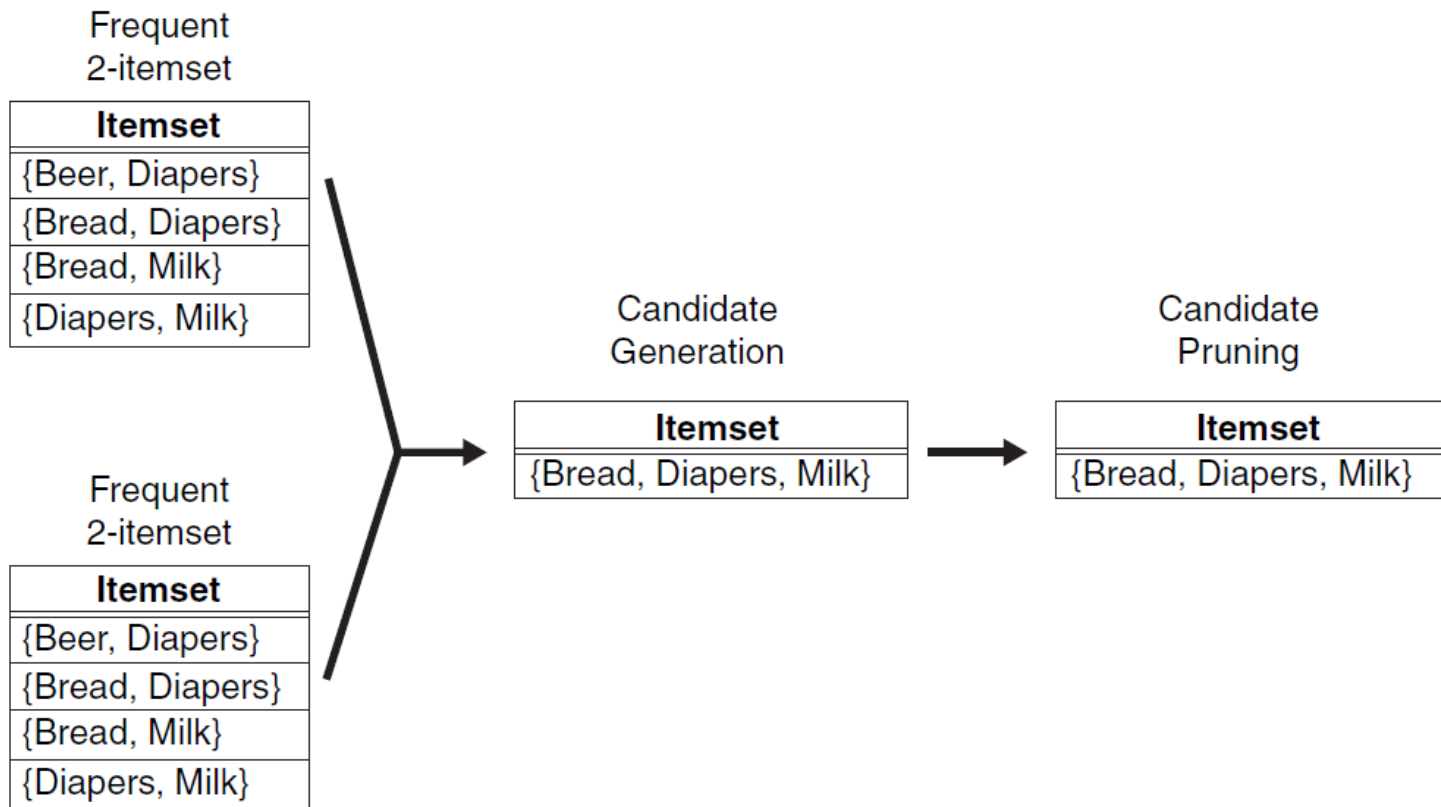


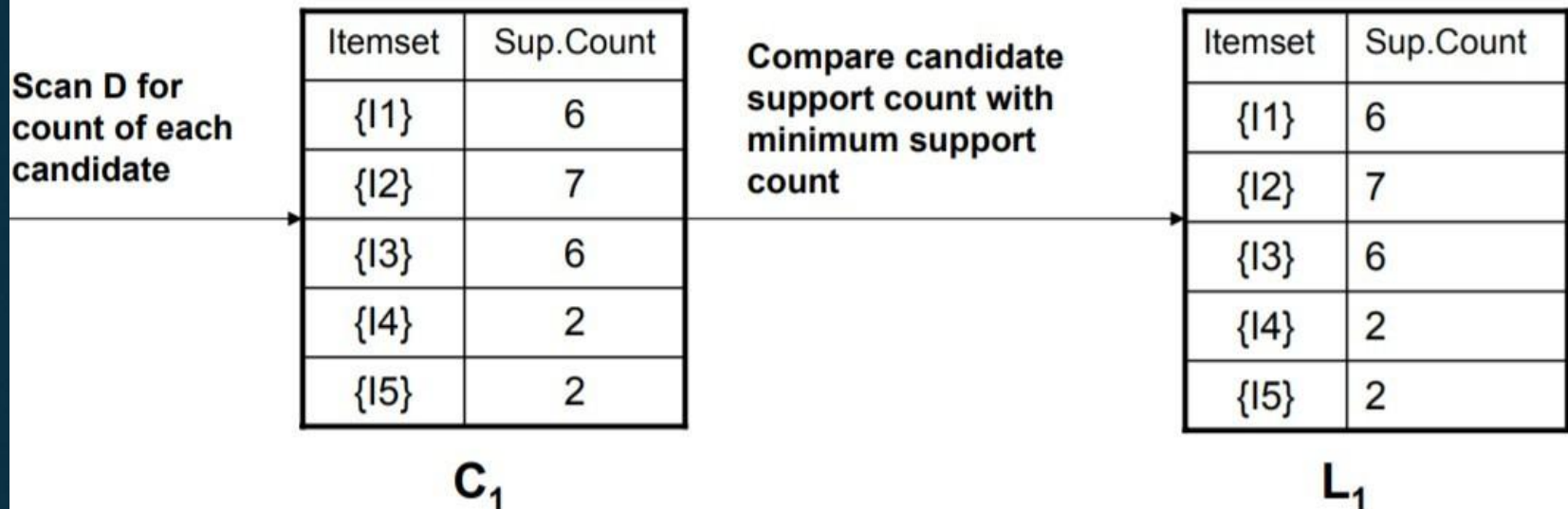
Figure 5.8. Generating and pruning candidate k -itemsets by merging pairs of frequent $(k-1)$ -itemsets.

The Apriori Algorithm: Example

TID	List of Items
T100	I1, I2, I5
T100	I2, I4
T100	I2, I3
T100	I1, I2, I4
T100	I1, I3
T100	I2, I3
T100	I1, I3
T100	I1, I2, I3, I5
T100	I1, I2, I3

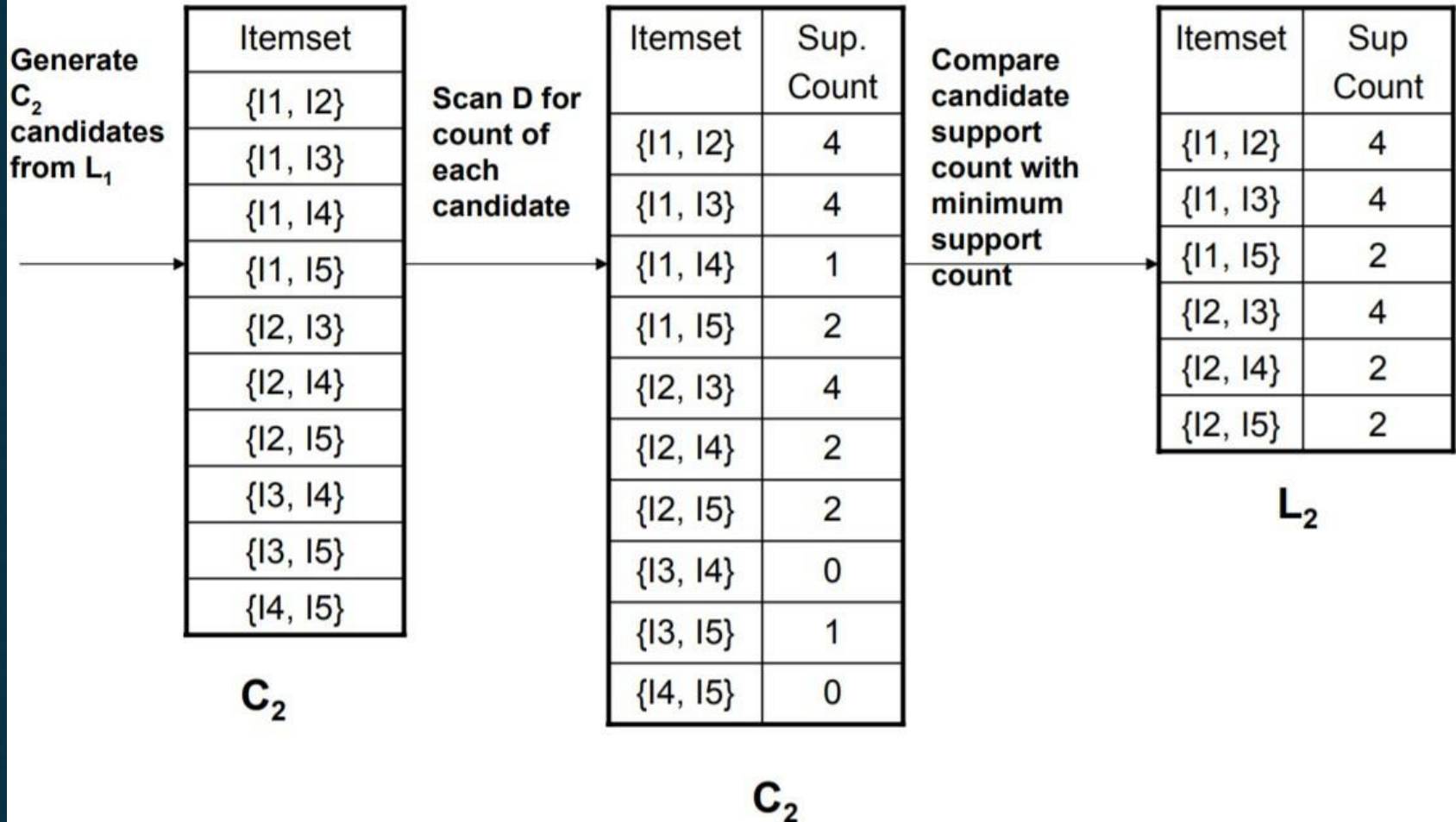
- Consider a database, D , consisting of 9 transactions.
- Suppose min. support count required is 2 (i.e. $\text{min_sup} = 2/9 = 22\%$)
- Let minimum confidence required is 70%.
- We have to first find out the frequent itemset using Apriori algorithm.
- Then, Association rules will be generated using min. support & min. confidence.

Step 1: Generating 1-itemset Frequent Pattern



- The set of frequent 1-itemsets, L_1 , consists of the candidate 1-itemsets satisfying minimum support.
- In the first iteration of the algorithm, each item is a member of the set of candidate.

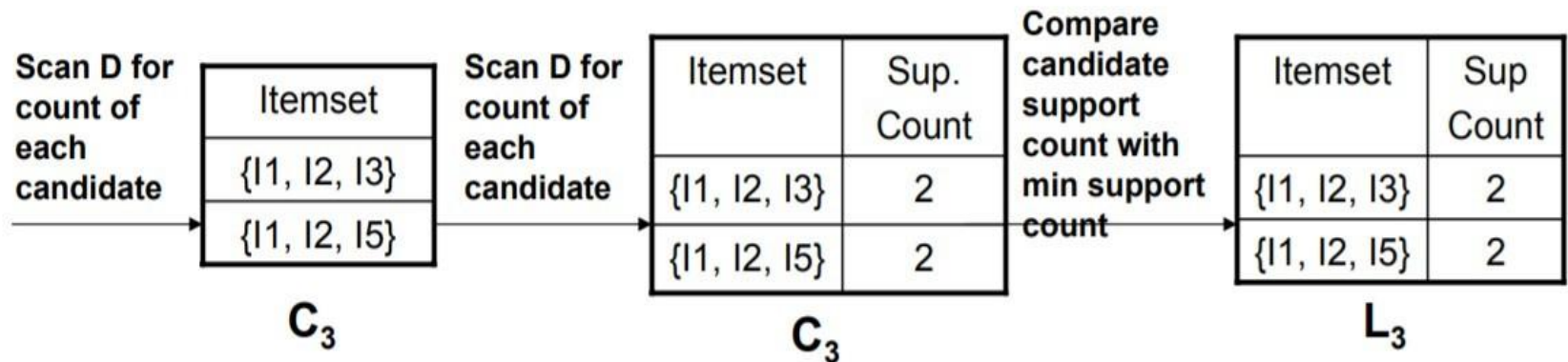
Step 2: Generating 2-itemset Frequent Pattern



Step 2: Generating 2-itemset Frequent Pattern

- To discover the set of frequent 2-itemsets, L_2 , the algorithm uses $L_1 \text{ Join } L_1$ to generate a candidate set of 2-itemsets, C_2 .
- Next, the transactions in D are scanned and the support count for each candidate itemset in C_2 is accumulated (as shown in the middle table).
- The set of frequent 2-itemsets, L_2 , is then determined, consisting of those candidate 2-itemsets in C_2 having minimum support.
- **Note:** We haven't used Apriori Property yet.

Step 3: Generating 3-itemset Frequent Pattern



- The generation of the set of candidate 3-itemsets, C_3 , involves use of the Apriori Property.
- In order to find C_3 , we compute $L_2 \text{ Join } L_2$.
- $C_3 = L_2 \text{ Join } L_2 = \{\{l1, l2, l3\}, \{l1, l2, l5\}, \{l1, l3, l5\}, \{l2, l3, l4\}, \{l2, l3, l5\}, \{l2, l4, l5\}\}$.
- Now, Join step is complete and Prune step will be used to reduce the size of C_3 . Prune step helps to avoid heavy computation due to large C_k .

Step 3: Generating 3-itemset Frequent Pattern

- Based on the **Apriori property** that all subsets of a frequent itemset must also be frequent, we can determine that four latter candidates cannot possibly be frequent. How ?
- For example , lets take **{I1, I2, I3}**. The 2-item subsets of it are {I1, I2}, {I1, I3} & {I2, I3}. Since all 2-item subsets of {I1, I2, I3} are members of L_2 , We will keep {I1, I2, I3} in C_3 .
- Lets take another example of **{I2, I3, I5}** which shows how the pruning is performed. The 2-item subsets are {I2, I3}, {I2, I5} & {I3, I5}.
- BUT, {I3, I5} is not a member of L_2 and hence it is not frequent **violating Apriori Property**. Thus We will have to remove {I2, I3, I5} from C_3 .
- Therefore, $C_3 = \{\{I1, I2, I3\}, \{I1, I2, I5\}\}$ after checking for all members of **result of Join operation** for **Pruning**.
- Now, the transactions in D are scanned in order to determine L_3 , **consisting of those candidates 3-itemsets in C_3 having minimum support.**

Step 4: Generating 4-itemset Frequent Pattern

- The algorithm uses $L_3 \text{ Join } L_3$ to generate a candidate set of 4-itemsets, C_4 . Although the join results in $\{\{I1, I2, I3, I5\}\}$, this itemset is pruned since its subset $\{\{I2, I3, I5\}\}$ is not frequent.
- Thus, $C_4 = \varnothing$, and algorithm terminates, having found all of the frequent items. This completes our Apriori Algorithm.
- What's Next ?
These frequent itemsets will be used to generate strong association rules (where strong association rules satisfy both minimum support & minimum confidence).

Step 5: Generating Association Rules from Frequent Itemsets

Procedure:

- For each frequent itemset I , generate all nonempty subsets of I .
- For every nonempty subset s of I , output the rule " $s \rightarrow (I-s)$ " if $\text{support_count}(I) / \text{support_count}(s) \geq \text{min_conf}$ where min_conf is minimum confidence threshold.

Back To Example:

We had $L = \{\{I1\}, \{I2\}, \{I3\}, \{I4\}, \{I5\}, \{I1, I2\}, \{I1, I3\}, \{I1, I5\}, \{I2, I3\}, \{I2, I4\}, \{I2, I5\}, \{I1, I2, I3\}, \{I1, I2, I5\}\}$.

- Lets take $I = \{I1, I2, I5\}$.
- Its all nonempty subsets are $\{I1, I2\}, \{I1, I5\}, \{I2, I5\}, \{I1\}, \{I2\}, \{I5\}$.

Step 5: Generating Association Rules from Frequent Itemsets

- Let minimum confidence threshold is , say 70%.
- The resulting association rules are shown below, each listed with its confidence.
 - R1: $I1 \wedge I2 \rightarrow I5$
 - Confidence = $sc\{I1, I2, I5\} / sc\{I1, I2\} = 2/4 = 50\%$
 - R1 is Rejected.
 - R2: $I1 \wedge I5 \rightarrow I2$
 - Confidence = $sc\{I1, I2, I5\} / sc\{I1, I5\} = 2/2 = 100\%$
 - R2 is Selected.
 - R3: $I2 \wedge I5 \rightarrow I1$
 - Confidence = $sc\{I1, I2, I5\} / sc\{I2, I5\} = 2/2 = 100\%$
 - R3 is Selected.

Step 5: Generating Association Rules from Frequent Itemsets

- R4: $I_1 \rightarrow I_2 \wedge I_5$
 - Confidence = $sc\{I_1, I_2, I_5\} / sc\{I_1\} = 2/6 = 33\%$
 - R4 is Rejected.
- R5: $I_2 \rightarrow I_1 \wedge I_5$
 - Confidence = $sc\{I_1, I_2, I_5\} / \{I_2\} = 2/7 = 29\%$
 - R5 is Rejected.
- R6: $I_5 \rightarrow I_1 \wedge I_2$
 - Confidence = $sc\{I_1, I_2, I_5\} / \{I_5\} = 2/2 = 100\%$
 - R6 is Selected.

In this way, We have found three strong association rules.

Frequent itemset generation part of the Apriori algorithm has two important characteristics

- **Level-wise algorithm**

It traverses the itemset lattice one level at a time, from frequent 1-itemsets to the maximum size of frequent itemset

- **Employs a generate-and-test strategy for finding frequent itemset**

- At each iteration, new candidate itemsets are generated from the frequent itemsets found in the previous iteration. The support for each candidate is then counted and tested against the minsup threshold

Exercise:

Consider following transactions:

Transaction	Items
T1	<u>1</u> , <u>2</u> ,13
T2	<u>2</u> , <u>3</u> ,14
T3	14, <u>5</u>
T4	<u>1</u> , <u>2</u> ,14
T5	<u>1</u> , <u>2</u> ,13,15
T6	<u>1</u> , <u>2</u> ,13,14

Generate frequent item sets Using Apriori Algorithm. Also generate association rules from it.

Given: Minimum support=3, Minimum Confidence= 60%

FP Growth Algorithm

Apriori: uses a generate-and-test approach – generates candidate itemsets and tests if they are frequent

- Generation of candidate itemsets is expensive (in both space and time)
- Support counting is expensive

FP-Growth: allows frequent itemset discovery without candidate itemset generation. Two step approach:

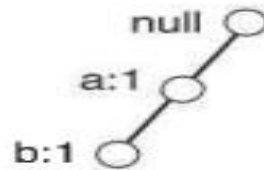
Step 1: Build a compact data structure called the FP-tree

Built using 2 passes over the data-set.

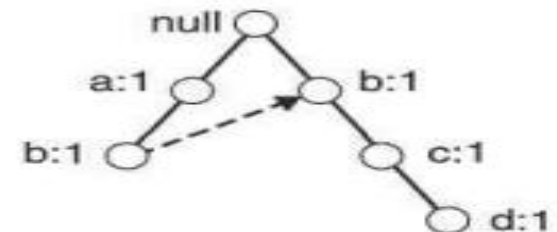
Step 2: Extracts frequent itemsets directly from the FP-tree

Construction of FP Tree

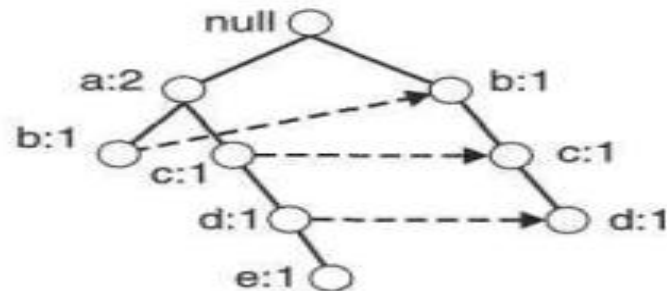
Transaction Data Set	
TID	Items
1	{a,b}
2	{b,c,d}
3	{a,c,d,e}
4	{a,d,e}
5	{a,b,c}
6	{a,b,c,d}
7	{a}
8	{a,b,c}
9	{a,b,d}
10	{b,c,e}



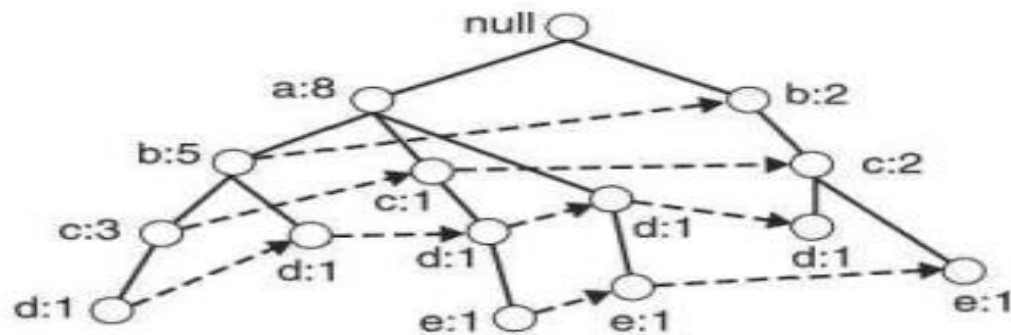
(i) After reading TID=1



(ii) After reading TID=2



(iii) After reading TID=3



(iv) After reading TID=10

Example 2:

Transaction ID	Items
T1	{ <u>E</u> ,K,M,N,O,Y}
T2	{ <u>D</u> , <u>E</u> ,K,N,O,Y}
T3	{ <u>A</u> ,E,K,M}
T4	{C, <u>K</u> ,M,U,Y}
T5	{C, <u>E</u> ,I,K,O,O}

- The frequency of each individual item is computed:

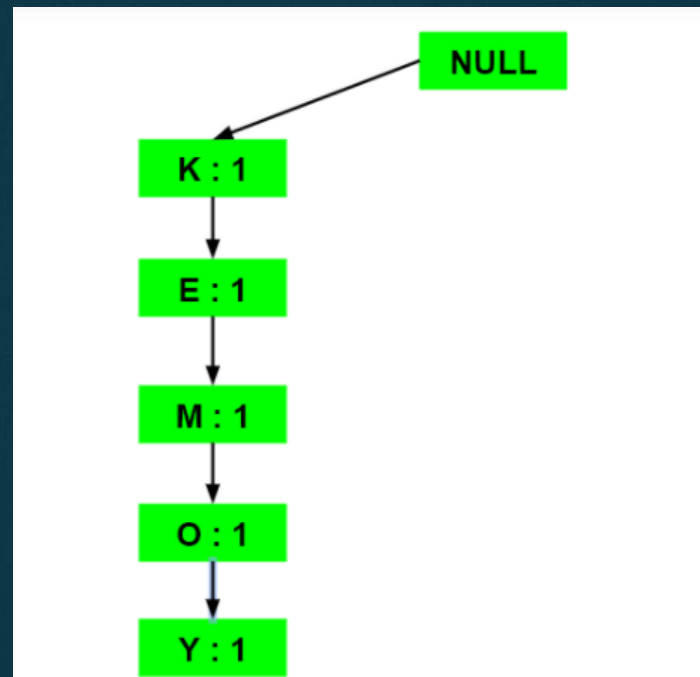
Item	Frequency
A	1
C	2
D	1
E	4
I	1
K	5
M	3
N	2
O	3
U	1
Y	3

- Let the minimum support be 3.
- A **Frequent Pattern** set is built which will contain all the elements whose frequency is greater than or equal to the minimum support.
- These elements are stored in descending order of their respective frequencies. After insertion of the relevant items, the set L looks like this:-
- $L = \{K : 5, E : 4, M : 3, O : 3, Y : 3\}$

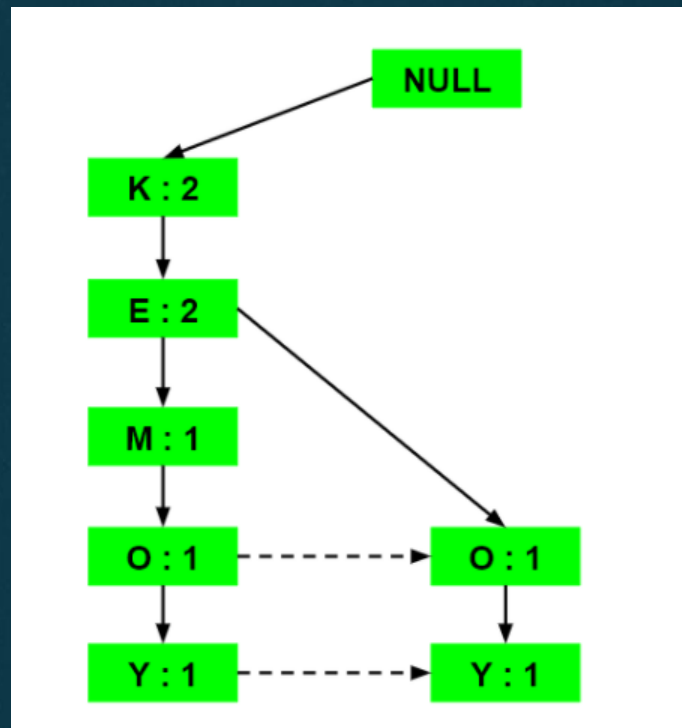
- Now, for each transaction, the respective Ordered-Item set is built.

Transaction ID	Items	Ordered-Item Set
T1	{E, <u>K</u> ,M,N,O,Y}	{ <u>K</u> ,E,M,O,Y}
T2	{D, <u>E</u> ,K,N,O,Y}	{ <u>K</u> ,E,O,Y}
T3	{A, <u>E</u> ,K,M}	{ <u>K</u> ,E,M}
T4	{C, <u>K</u> ,M,U,Y}	{ <u>K</u> ,M,Y}
T5	{C, <u>E</u> ,I,K,O,O}	{ <u>K</u> ,E,O}

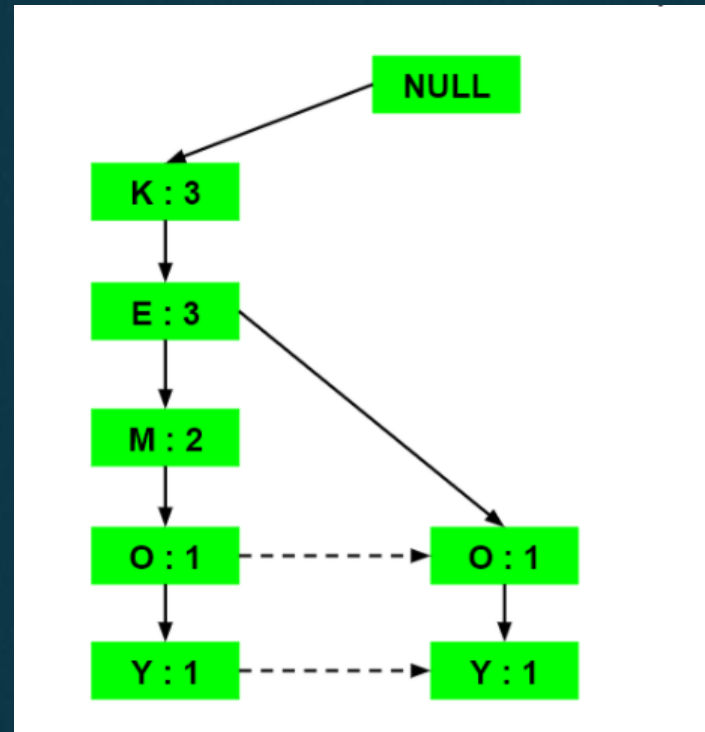
- a) Inserting the set {K, E, M, O, Y}:



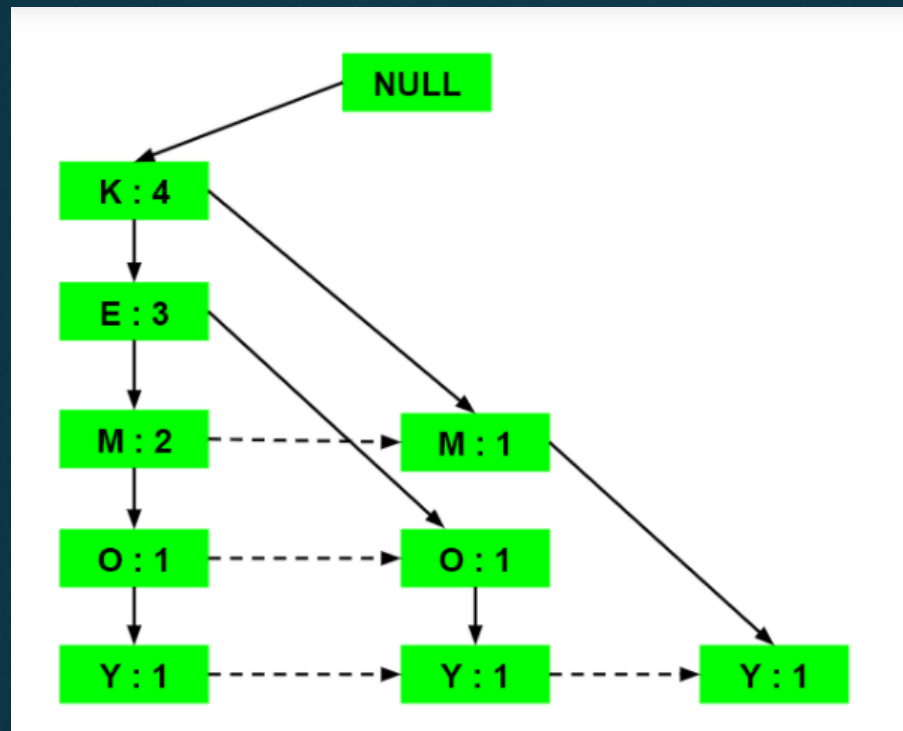
- b) Inserting the set {K, E, O, Y}:



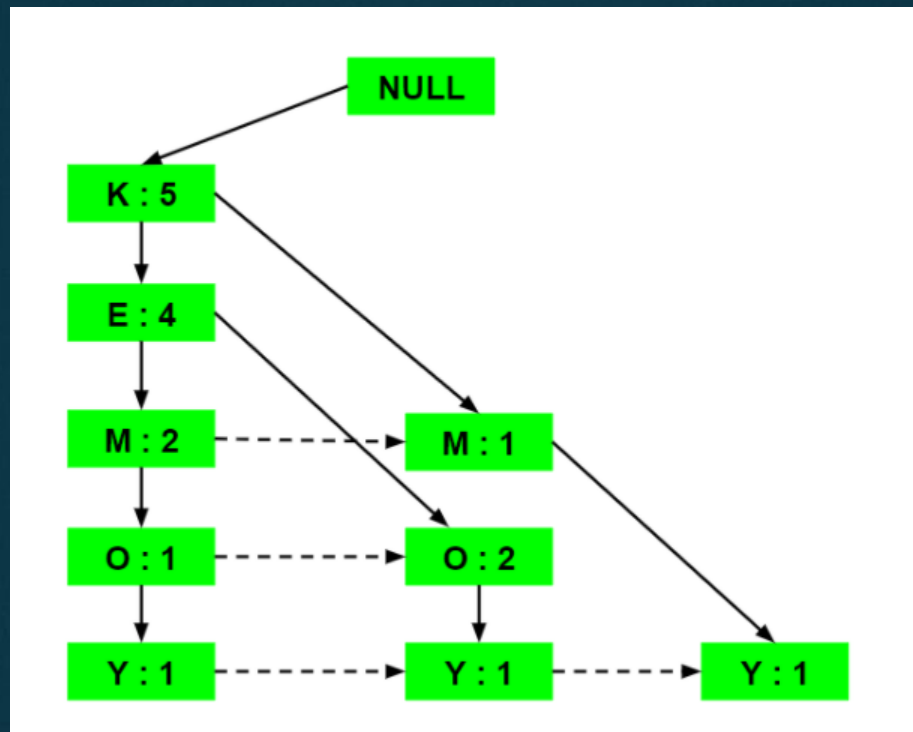
► c) Inserting the set {K, E, M}



- d) Inserting the set {K, M, Y}:



- e) Inserting the set {K, E, O}:



- Now, for each item, the Conditional Pattern Base is computed which is path labels of all the paths which lead to any node of the given item in the frequent-pattern tree.

Items	Conditional Pattern Base
Y	{{ <u>K</u> ,E,M,O : 1}, {K,E,O : 1}, {K,M : 1}}
O	{{ <u>K</u> ,E,M : 1}, {K,E : 2}}
M	{{ <u>K</u> ,E : 2}, {K : 1}}
E	{ <u>K</u> : 4}
K	

- Now for each item the Conditional Frequent Pattern Tree is built. It is done by taking the set of elements which is common in all the paths in the Conditional Pattern Base of that item and calculating it's support count by summing the support counts of all the paths in the Conditional Pattern Base

Items	Conditional Pattern Base	Conditional Frequent Pattern Tree
Y	{{ <u>K,E</u> ,M,O : 1}, {K,E,O : 1}, {K,M : 1}}	{ <u>K</u> : 3}
O	{{ <u>K,E</u> ,M : 1}, {K,E : 2}}	{ <u>K,E</u> : 3}
M	{{ <u>K,E</u> : 2}, {K : 1}}	{ <u>K</u> : 3}
E	{ <u>K</u> : 4}	{ <u>K</u> : 4}
K		

Items	Frequent Pattern Generated
Y	{< <u>K</u> ,Y : 3>}
O	{< <u>K</u> ,O : 3>, <E,O : 3>, <E,K,O : 3>}
M	{< <u>K</u> ,M : 3>}
E	{< <u>E</u> ,K : 3>}
K	

- From the Conditional Frequent Pattern tree, the **Frequent Pattern rules** are generated by pairing the items of the Conditional Frequent Pattern Tree set to the corresponding to the item

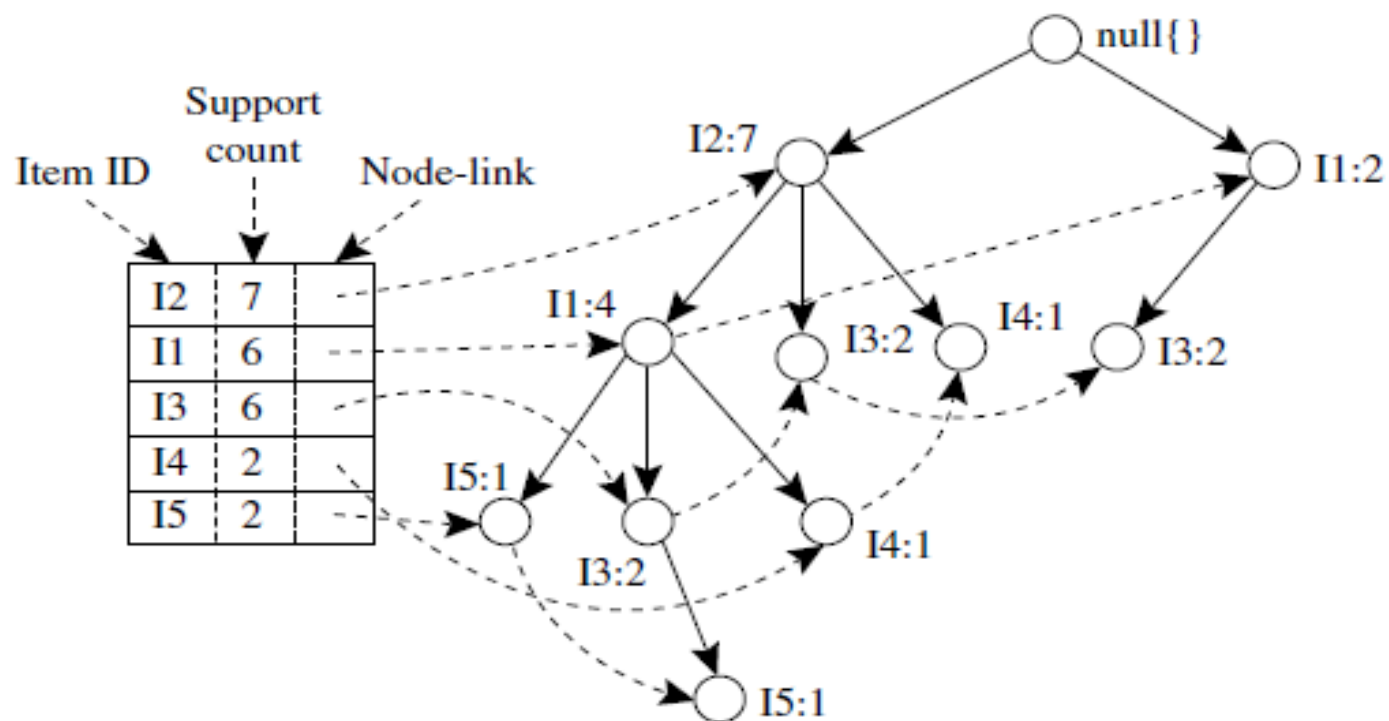
Items	Frequent Pattern Generated
Y	{< <u>K</u> ,Y : 3>}
O	{< <u>K</u> ,O : 3>, <E,O : 3>, <E,K,O : 3>}
M	{< <u>K</u> ,M : 3>}
E	{<E, <u>K</u> : 3>}
K	

FP-Tree Representation

- Compressed representation of the input data
- Constructed by reading the data set one transaction at a time and mapping each transaction onto a path in the FP-tree
- As different transactions can have several items in common, their paths may overlap
- more the paths overlap with one another, the more compression we can achieve
- If the size of the F.P-tree is small enough to fit into main memory, this will allow us to extract frequent itemsets directly from the structure in memory instead of making repeated passes over the data stored on disk

Transactional Data for an *AllElectronics* Branch

<i>TID</i>	<i>List of item_IDs</i>
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3



Mining the FP-Tree by Creating Conditional (Sub-)Pattern Bases

Item	Conditional Pattern Base	Conditional FP-tree	Frequent Patterns Generated
I5	{ {I2, I1: 1}, {I2, I1, I3: 1} }	$\langle I2: 2, I1: 2 \rangle$	{I2, I5: 2}, {I1, I5: 2}, {I2, I1, I5: 2}
I4	{ {I2, I1: 1}, {I2: 1} }	$\langle I2: 2 \rangle$	{I2, I4: 2}
I3	{ {I2, I1: 2}, {I2: 2}, {I1: 2} }	$\langle I2: 4, I1: 2 \rangle, \langle I1: 2 \rangle$	{I2, I3: 4}, {I1, I3: 4}, {I2, I1, I3: 2}
I1	{ {I2: 4} }	$\langle I2: 4 \rangle$	{I2, I1: 4}